

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 01 · LECTURE NOTES

From Planning to Learning

Two substitutions, and everything that follows.

MODULE OUTCOMES

- State the two differences from CSCE 625's MDP.
- Explain sample inefficiency and where it comes from.
- Distinguish model-free from model-based approaches.
- Classify reinforcement learning settings by what access you have.
- Decide whether a problem calls for learning at all.

1 The two substitutions

REINFORCEMENT LEARNING IS CSCE 625'S MDP WITH TWO THINGS TAKEN AWAY

The Bellman equation is unchanged. $V^*(s) = \max_a \sum_s' P(s'|s,a)[R + \gamma V^*(s')]$ is still exactly the thing being solved, and **recognising that is the most useful single fact for navigating this course.**

Substitution one: you do not know P or R. So you cannot evaluate the sum over s' — you must estimate it by acting in the world and observing what happens, which means every update is based on samples rather than on the distribution.

Substitution two: the state space is far too large to tabulate. So V or Q becomes a *function approximator* — in practice a neural network (CSCE 636) — which generalises across states it has never visited and, in doing so, introduces errors that feed back into its own targets.

Every difficulty in this course traces to one of those two substitutions. Sampling causes inefficiency and makes exploration an open problem; approximation causes instability and removes the convergence guarantees. Keeping the attribution clear is what makes the field comprehensible rather than a list of algorithms — when a method is introduced, ask which substitution's damage it is repairing.

	No model (sampling)	No table (approximation)
What it causes	Sample inefficiency, and exploration becomes a problem that must be solved rather than assumed.	Instability, and generalisation across states — which is both the benefit and the hazard.
Modules addressing it	02, 03, 08	04, 05, 06, 07

	No model (sampling)	No table (approximation)
Guarantees	Tabular methods converge given enough visits to every state-action pair — a real theorem.	None, in general. See the note below.
The standard fixes	Better exploration (Module 02); learn a model to reuse experience (Module 08).	Experience replay, target networks, trust regions, and clipping (Modules 04, 06).
What remains unfixed	Millions of environment steps for tasks humans learn in minutes.	Seed variance large enough to swamp method differences (Module 12).

Note that the second column has no guarantees row worth stating. Tabular reinforcement learning converges and deep reinforcement learning is an empirical discipline, and this course says so throughout rather than once. **It is not a criticism** — CSCE 636's deep networks have no guarantees either and are extremely useful — **but it does change what a result means and how it should be reported**, which is Module 12's subject.

2 Sample inefficiency

THE SAMPLE COUNTS ARE THE FIRST THING TO INTERNALISE

Atari at human level: tens of millions of frames for the original deep Q-network, and still millions for modern sample-efficient methods. **A human reaches comparable performance in a couple of hours — roughly a hundred thousand frames**, and brings priors about objects, gravity, and intent that the agent does not have.

Continuous control: millions of steps for tasks a hand-written controller performs immediately — and the hand-written controller is frequently better, which Module 13 reports honestly.

So reinforcement learning is viable where samples are cheap — a fast simulator, a game, a recommender with enormous traffic — **and very difficult where they are not**, which includes most of the physical world, most medical settings, and anything where a bad action has a real cost.

That single fact determines the field's shape. The well-known successes are in games and simulation; the robotics work is largely about transferring from simulation (domain randomisation) or about learning a model to reduce the real-sample requirement (Module 08). Reading the literature with the sample count in mind explains most of what the field chooses to work on.

3 The settings

What you have access to	The setting	The approach
The transition model and reward function	Planning.	CSCE 625 Module 10's value or policy iteration. Not this course.

What you have access to	The setting	The approach
A fast simulator you can reset and query freely	On-policy reinforcement learning.	PPO is the default (Modules 05–06).
Interaction that is slow or costly	Off-policy reinforcement learning.	SAC or TD3 for sample efficiency (Module 07), or model-based (Module 08).
A fixed dataset of logged interactions and no further access	Offline reinforcement learning.	Conservative methods — CQL, IQL (Module 09), and naive off-policy learning fails here for a specific reason.
Expert demonstrations	Imitation learning.	Behaviour cloning, DAgger, inverse reinforcement learning (Module 09).
A notion of good behaviour you cannot write as a reward	Preference-based learning.	Learn a reward model from comparisons — RLHF (Module 10).

The first row is the one people skip. If you have a model, or can build an approximate one, computing is better than learning — it is faster, it has guarantees, it is debuggable, and it needs no samples at all. And building an approximate model is frequently easier than people assume, especially for engineered systems where the dynamics are documented. Starting this course by naming the setting in which it does not apply is deliberate.

MODEL-FREE AND MODEL-BASED, AND WHY BOTH EXIST

Model-free: learn a value function or a policy directly from experience, with no explicit representation of the dynamics at all. Conceptually simpler, far more widely used, and the subject of Modules 03 through 07.

Model-based: learn P and R from experience, then either plan with them (CSCE 625 Module 10, or Module 05's tree search in that course) or use them to generate synthetic experience to train a model-free learner on. Module 08.

Model-based is dramatically more sample-efficient, because a learned model can be queried millions of times for the cost of one real environment step — and sample efficiency is §2's binding constraint, so this is not a minor advantage.

And it is harder, because model errors compound over a rollout. A policy optimised against a flawed model exploits the flaw — it finds the state where the model is most wrong and goes there, because that is where the model promises the most reward. This is CSCE 669 Module 12 §4's warning in its sharpest available form, and it is why model-based reinforcement learning took so long to work: the obvious approach fails for a reason that is structural rather than incidental.

4 Should you use reinforcement learning at all

- **Yes, when:** decisions are sequential, consequences are delayed, you have a simulator or cheap interaction, no good hand-written policy exists, **and you can actually specify a reward. All five, not three of five.**
- **No, if a scripted or searched policy works. It will be faster at runtime, debuggable, and tunable by a designer** — CSCE 625 Module 01 §4's four properties, all of which matter more in production than the last few percent of performance.
- **No, if you have the model. Plan instead. Computing beats learning whenever you can compute,** and §3's first row is there for this reason.
- **No, if the decisions are independent of each other. That is supervised learning,** which is far easier, far better understood, and has actual generalisation theory (CSCE 633). **A great deal of work framed as reinforcement learning is a contextual bandit at most,** and a bandit is much easier than a full MDP (Module 02 §1).
- **And no, if you cannot specify the reward. Module 10 is about how badly that goes, and it goes badly** — an agent given an imperfect reward will find the imperfection, reliably and creatively.
- **'No' is a respectable answer, and this course's second project accepts it with evidence. Knowing when not to use a method is part of knowing the method,** and it is the part the literature is least helpful about.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sutton & Barto — Reinforcement Learning, chapter 1 (free PDF)	The field's own framing , and chapters 3–4 are the CSCE 625 Module 10 material this course builds on.
OpenAI Spinning Up — Introduction and Key Concepts (free)	The clearest statement of §1's framing available , with the taxonomy of §3's table.
Berkeley CS285 — lecture 1 and the course overview (free)	The graduate framing , including an honest treatment of §2's sample-efficiency problem.
Tsividis et al. — Human Learning in Atari (free)	The human comparison of §2, measured — and the analysis of which priors humans bring.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write the Bellman equation** and mark which terms you do not have access to in the reinforcement learning setting.
- 2 **Solve a small MDP by value iteration** (CSCE 625 M10), then **solve the same MDP by Q-learning** and report the sample count needed to match.
- 3 **Report the ratio**, and explain which substitution it measures.
- 4 **Classify three problems you care about** against §3's table.
- 5 **For each, answer §4's five questions** and record whether reinforcement learning is appropriate.
- 6 **Find a published reinforcement learning result** and locate its total environment step count. Report it.
- 7 **Estimate how long that would take** at one step per second of real-world interaction.
- 8 **Build a scripted baseline** for one of your tasks and record its performance before training anything.
- 9 **Set up seed control and logging** for the five quantities in the tooling list, before Module 02.
- 10 **Write down, now, what you expect reinforcement learning to be good at.** Keep it and revisit after Module 13.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What two things does reinforcement learning remove from CSCE 625's MDP?
- 2 What does each removal cause, and which modules address each?
- 3 Which has convergence guarantees and which does not?
- 4 Give the Atari sample counts for an agent and for a human.
- 5 How does sample efficiency determine the field's shape?
- 6 Name six settings and the approach for each.
- 7 Why is the planning row first?
- 8 Compare model-free and model-based, including the failure mode.
- 9 Give five conditions for reinforcement learning being appropriate.